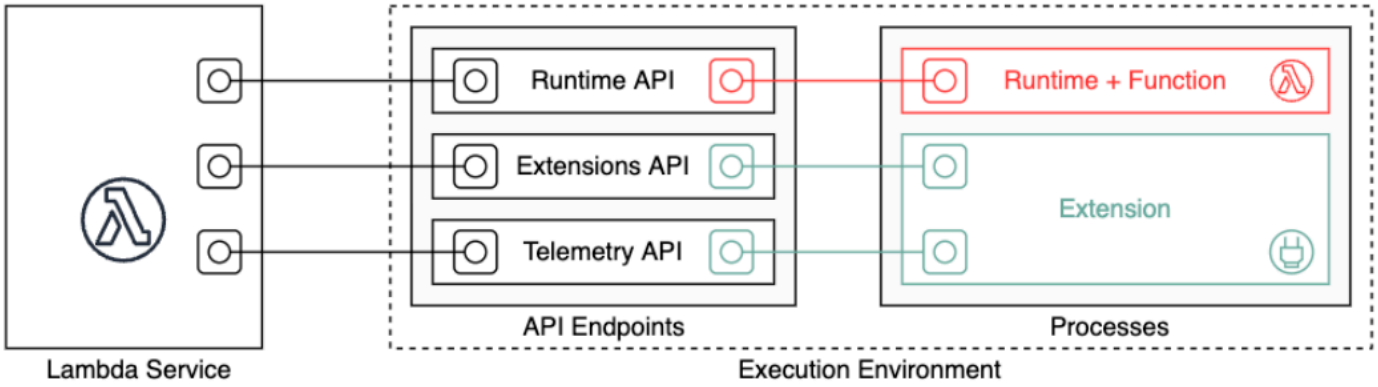
Using the Lambda Extensions API to create extensions

Lambda function authors use extensions to integrate Lambda with their preferred tools for monitoring, observability, security, and governance. Function authors can use extensions from AWS, AWS Partners, and open-source projects. For more information on using extensions, see Introducing AWS Lambda Extensions on the AWS Compute Blog. This section describes how to use the Lambda Extensions API, the Lambda execution environment lifecycle, and the Lambda Extensions API reference.



As an extension author, you can use the Lambda Extensions API to integrate deeply into the Lambda execution environment. Your extension can register for function and execution environment lifecycle events. In response to these events, you can start new processes, run logic, and control and participate in all phases of the Lambda lifecycle: initialization, invocation, and shutdown. In addition, you can use the Runtime Logs API to receive a stream of logs.

An extension runs as an independent process in the execution environment and can continue to run after the function invocation is fully processed. Because extensions run as processes, you can write them in a different language than the function. We recommend that you implement extensions using a compiled language. In this case, the extension is a self-contained binary that is compatible with supported runtimes. All Lambda runtimes support extensions. If you use a non-compiled language, ensure that you include a compatible runtime in the extension.

Lambda also supports *internal extensions*. An internal extension runs as a separate thread in the runtime process. The runtime starts and stops the internal extension. An alternative way to integrate with the Lambda environment is to use language-specific environment variables and wrapper scripts. You can use these to configure the runtime environment and modify the startup behavior of the runtime process.

You can add extensions to a function in two ways. For a function deployed as a .zip file archive, you deploy your extension as a layer. For a function defined as a container image, you add the extensions to your container image.

> **Note**
>
> For example extensions and wrapper scripts, see AWS Lambda Extensions on the AWS Samples GitHub repository.

Topics

- Lambda execution environment lifecycle
- Extensions API reference

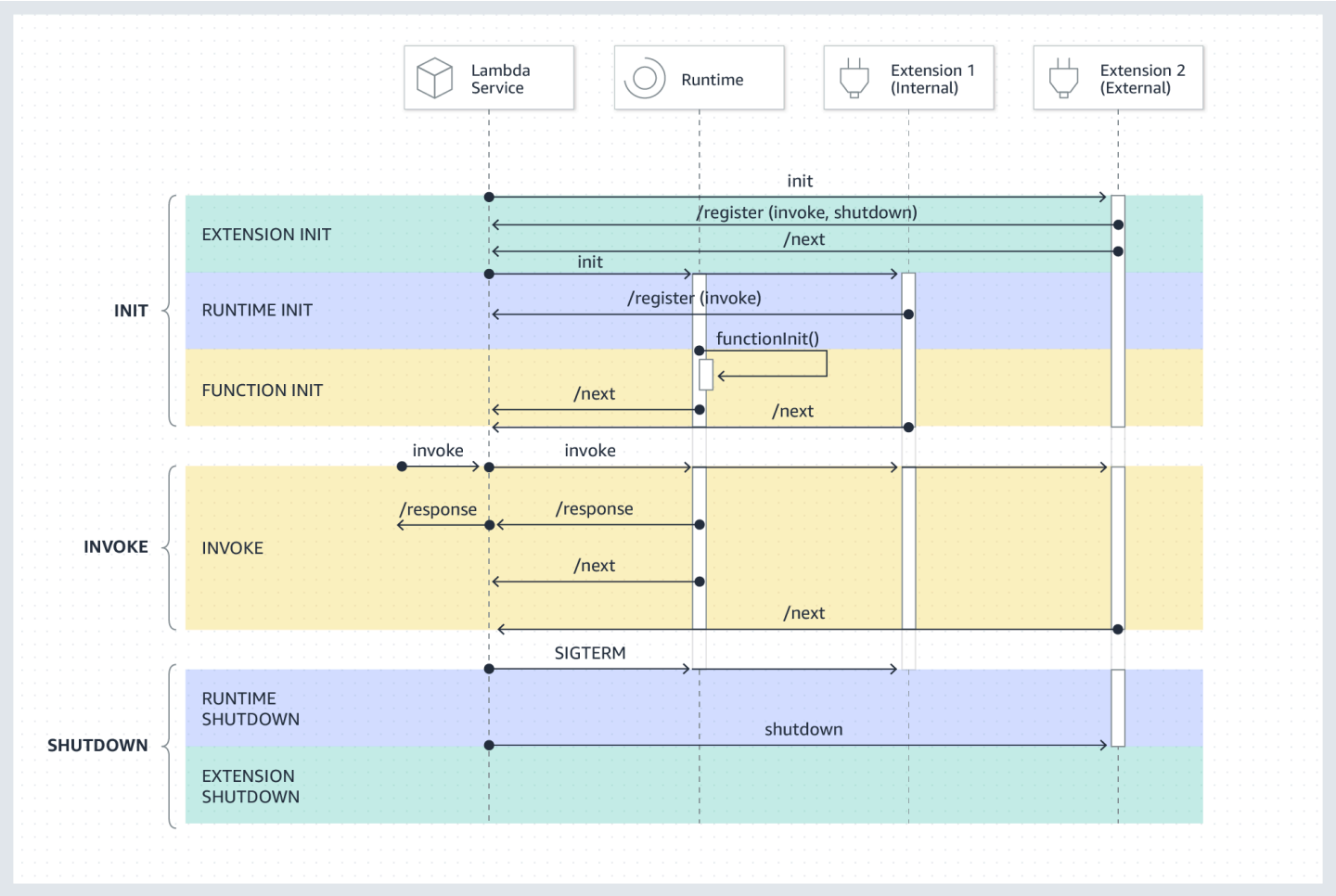
Lambda execution environment lifecycle

The lifecycle of the execution environment includes the following phases:

- **`Init`**: In this phase, Lambda creates or unfreezes an execution environment with the configured resources, downloads the code for the function and all layers, initializes any extensions, initializes the runtime, and then runs the function's initialization code (the code outside the main handler). The `Init` phase happens either during the first invocation, or in advance of function invocations if you have enabled provisioned concurrency.

  The `Init` phase is split into three sub-phases: `Extension init`, `Runtime init`, and `Function init`. These sub-phases ensure that all extensions and the runtime complete their setup tasks before the function code runs.
- **`Invoke`**: In this phase, Lambda invokes the function handler. After the function runs to completion, Lambda prepares to handle another function invocation.
- **`Shutdown`**: This phase is triggered if the Lambda function does not receive any invocations for a period of time. In the `Shutdown` phase, Lambda shuts down the runtime, alerts the extensions to let them stop cleanly, and then removes the environment. Lambda sends a `Shutdown` event to each extension, which tells the extension that the environment is about to be shut down.

Each phase starts with an event from Lambda to the runtime and to all registered extensions. The runtime and each extension signal completion by sending a `Next` API request. Lambda freezes the execution environment when each process has completed and there are no pending events.



Topics

- Init phase
- Invoke phase
- Shutdown phase
- Permissions and configuration
- Failure handling
- Troubleshooting extensions

Init phase

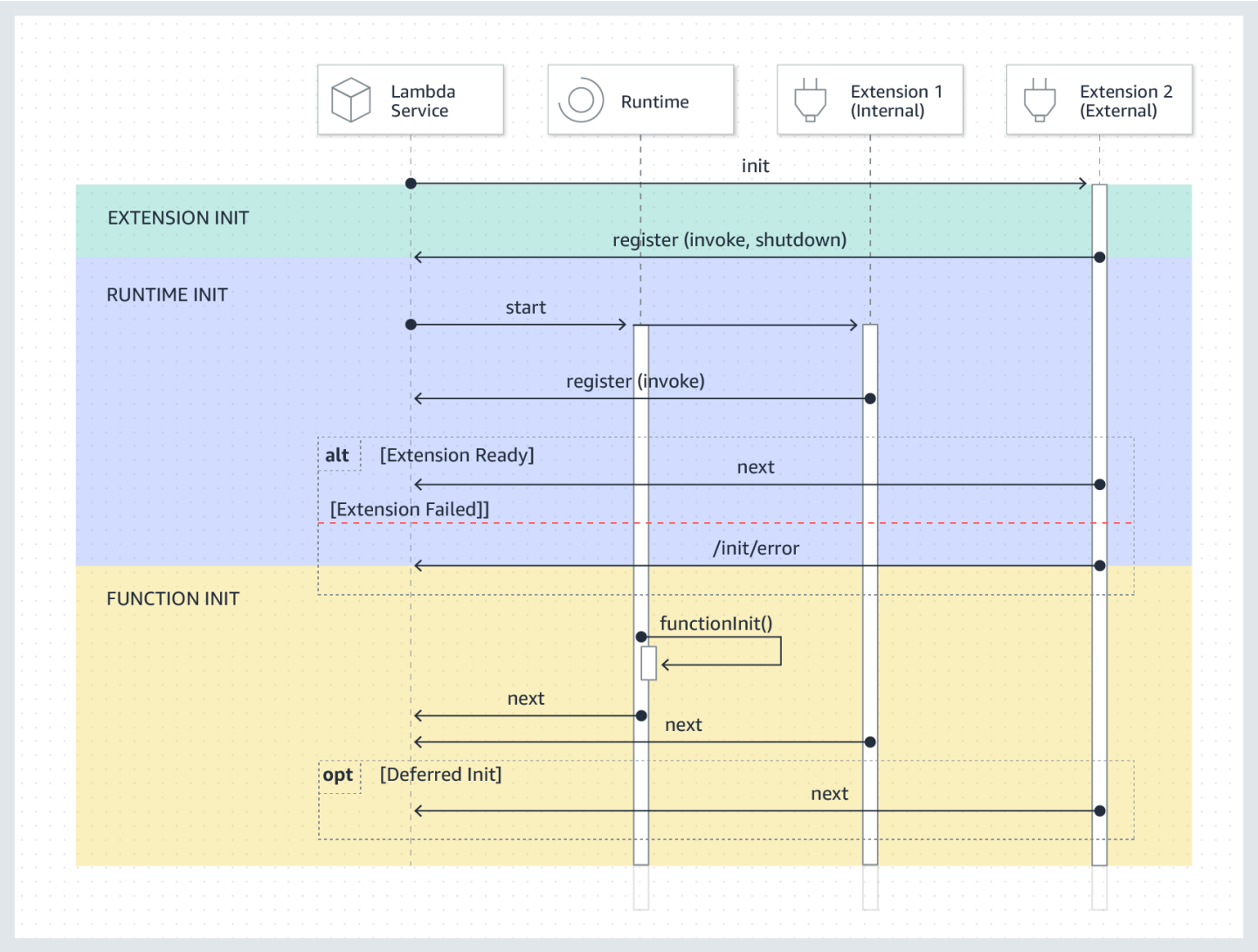
During the `Extension init` phase, each extension needs to register with Lambda to receive events. Lambda uses the full file name of the extension to validate that the extension has completed the bootstrap sequence. Therefore, each `Register` API call must include the `Lambda-Extension-Name` header with the full file name of the extension.

You can register up to 10 extensions for a function. This limit is enforced through the `Register` API call.

After each extension registers, Lambda starts the `Runtime init` phase. The runtime process calls `functionInit` to start the `Function init` phase.

The `Init` phase completes after the runtime and each registered extension indicate completion by sending a `Next` API request.

> **Note**
>
> Extensions can complete their initialization at any point in the `Init` phase.



Invoke phase

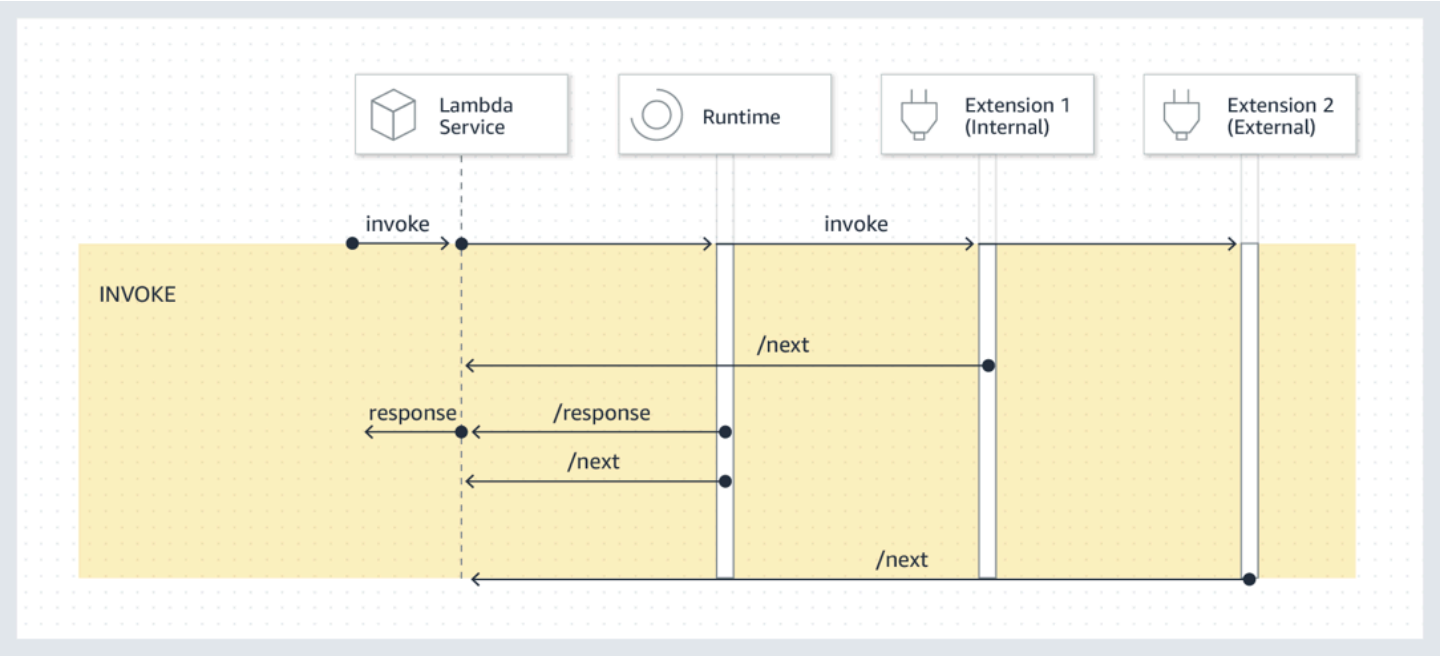
When a Lambda function is invoked in response to a `Next` API request, Lambda sends an `Invoke` event to the runtime and to each extension that is registered for the `Invoke` event.

> **Note**
>
> **Lambda Managed Instances:** Extensions for Lambda Managed Instances functions cannot register for the `Invoke` event. Because Lambda Managed Instances supports concurrent invocations within a single execution environment, the `Invoke` event is not supported. Extensions can only register for the `Shutdown` event. If you need to track when invocations start and finish, use the `platform.report` platform event through the Telemetry API.

During the invocation, external extensions run in parallel with the function. They also continue running after the function has completed. This enables you to capture diagnostic information or to send logs, metrics, and traces to a location of your choice.

After receiving the function response from the runtime, Lambda returns the response to the client, even if extensions are still running.

The `Invoke` phase ends after the runtime and all extensions signal that they are done by sending a `Next` API request.



Event payload: The event sent to the runtime (and the Lambda function) carries the entire request, headers (such as `RequestId`), and payload. The event sent to each extension contains metadata that describes the event content. This lifecycle event includes the type of the event, the time that the function times out (`deadlineMs`), the `requestId`, the invoked function's Amazon Resource Name (ARN), and tracing headers.

Extensions that want to access the function event body can use an in-runtime SDK that communicates with the extension. Function developers use the in-runtime SDK to send the payload to the extension when the function is invoked.

Here is an example payload:

```
{
    "eventType": "INVOKE",
```

```
    "deadlineMs": 676051,
    "requestId": "3da1f2dc-3222-475e-9205-e2e6c6318895",
    "invokedFunctionArn": "arn:aws:lambda:us-
east-1:123456789012:function:ExtensionTest",
    "tracing": {
        "type": "X-Amzn-Trace-Id",
        "value":
 "Root=1-5f35ae12-0c0fec141ab77a00bc047aa2;Parent=2be948a625588e32;Sampled=1"
    }
 }
```

Duration limit: The function's timeout setting limits the duration of the entire `Invoke` phase. For example, if you set the function timeout as 360 seconds, the function and all extensions need to complete within 360 seconds. Note that there is no independent post-invoke phase. The duration is the total time it takes for your runtime and all your extensions' invocations to complete and is not calculated until the function and all extensions have finished running.

Performance impact and extension overhead: Extensions can impact function performance. As an extension author, you have control over the performance impact of your extension. For example, if your extension performs compute-intensive operations, the function's duration increases because the extension and the function code share the same CPU resources. In addition, if your extension performs extensive operations after the function invocation completes, the function duration increases because the `Invoke` phase continues until all extensions signal that they are completed.

> **Note**
>
> Lambda allocates CPU power in proportion to the function's memory setting. You might see increased execution and initialization duration at lower memory settings because the function and extension processes are competing for the same CPU resources. To reduce the execution and initialization duration, try increasing the memory setting.

To help identify the performance impact introduced by extensions on the `Invoke` phase, Lambda outputs the `PostRuntimeExtensionsDuration` metric. This metric measures the cumulative time spent between the runtime `Next` API request and the last extension `Next` API request. To measure the increase in memory used, use the `MaxMemoryUsed` metric. For more information about function metrics, see Using CloudWatch metrics with Lambda.

Function developers can run different versions of their functions side by side to understand the impact of a specific extension. We recommend that extension authors publish expected resource consumption to make it easier for function developers to choose a suitable extension.

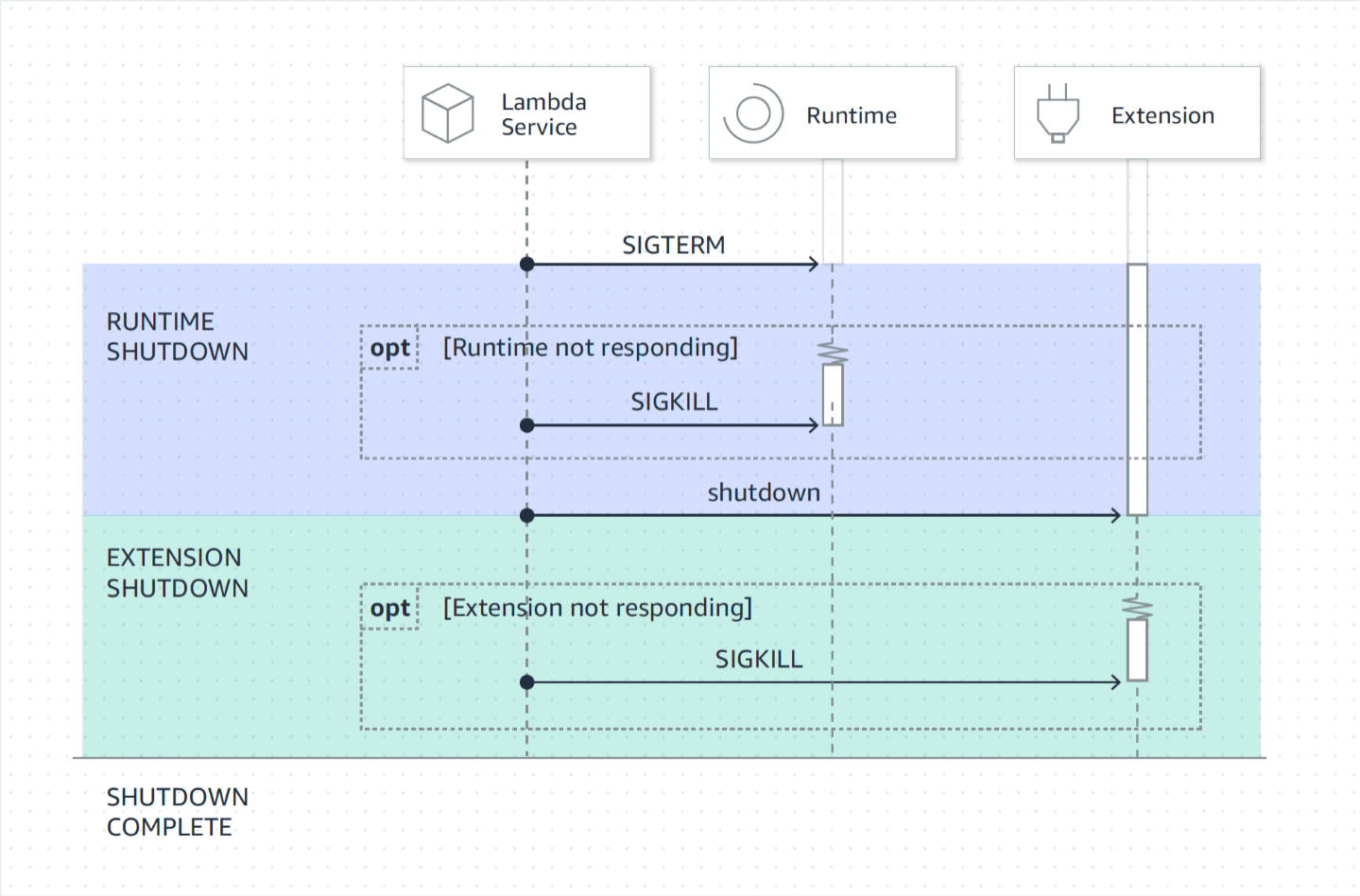
Shutdown phase

When Lambda is about to shut down the runtime, it sends a `Shutdown` to each registered external extension. Extensions can use this time for final cleanup tasks. The `Shutdown` event is sent in response to a `Next` API request.

Duration limit: The maximum duration of the `Shutdown` phase depends on the configuration of registered extensions:

- 0 ms – A function with no registered extensions
- 500 ms – A function with a registered internal extension
- 2,000 ms – A function with one or more registered external extensions

If the runtime or an extension does not respond to the `Shutdown` event within the limit, Lambda ends the process using a `SIGKILL` signal.



Event payload: The `Shutdown` event contains the reason for the shutdown and the time remaining in milliseconds.

The `shutdownReason` includes the following values:

- SPINDOWN – Normal shutdown
- TIMEOUT – Duration limit timed out
- FAILURE – Error condition, such as an `out-of-memory` event

```
{
  "eventType": "SHUTDOWN",
  "shutdownReason": "reason for shutdown",
  "deadlineMs": "the time and date that the function times out in Unix time
 milliseconds"
}
```

Permissions and configuration

Extensions run in the same execution environment as the Lambda function. Extensions also share resources with the function, such as CPU, memory, and `/tmp` disk storage. In addition, extensions use the same AWS Identity and Access Management (IAM) role and security context as the function.

File system and network access permissions: Extensions run in the same file system and network name namespace as the function runtime. This means that extensions need to be compatible with the associated operating system. If an extension requires any additional outbound network traffic rules, you must apply these rules to the function configuration.

> **Note**
>
> Because the function code directory is read-only, extensions cannot modify the function code.

Environment variables: Extensions can access the function's environment variables, except for the following variables that are specific to the runtime process:

- `AWS_EXECUTION_ENV`
- `AWS_LAMBDA_LOG_GROUP_NAME`
- `AWS_LAMBDA_LOG_STREAM_NAME`
- `AWS_XRAY_CONTEXT_MISSING`
- `AWS_XRAY_DAEMON_ADDRESS`
- `LAMBDA_RUNTIME_DIR`
- `LAMBDA_TASK_ROOT`
- `_AWS_XRAY_DAEMON_ADDRESS`
- `_AWS_XRAY_DAEMON_PORT`
- `_HANDLER`

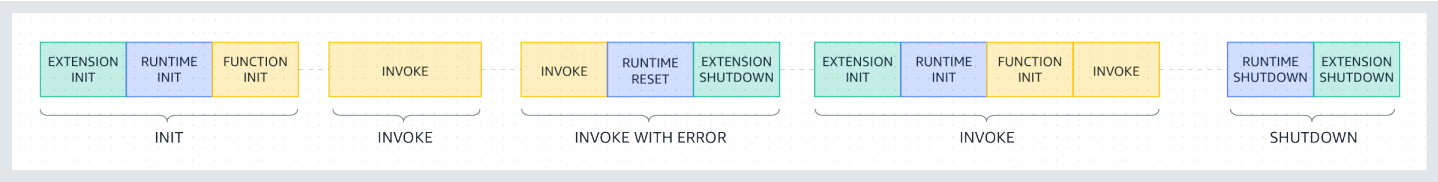
> **Note**
>
> **Detecting Lambda Managed Instances:** Extensions can check the `AWS_LAMBDA_INITIALIZATION_TYPE` environment variable to determine if they are running on Lambda Managed Instances versus Lambda (default) functions. This is the recommended method for extensions to adapt their behavior based on the execution environment type.

Failure handling

Initialization failures: If an extension fails, Lambda restarts the execution environment to enforce consistent behavior and to encourage fail fast for extensions. Also, for some customers, the extensions must meet mission-critical needs such as logging, security, governance, and telemetry collection.

Invoke failures (such as out of memory, function timeout): Because extensions share resources with the runtime, memory exhaustion affects them. When the runtime fails, all extensions and the runtime itself participate in the `Shutdown` phase. In addition, the runtime is restarted—either automatically as part of the current invocation, or via a deferred re-initialization mechanism.

If there is a failure (such as a function timeout or runtime error) during `Invoke`, the Lambda service performs a reset. The reset behaves like a `Shutdown` event. First, Lambda shuts down the runtime, then it sends a `Shutdown` event to each registered external extension. The event includes the reason for the shutdown. If this environment is used for a new invocation, the extension and runtime are re-initialized as part of the next invocation.



For a more detailed explanation of the previous diagram, see Failures during the invoke phase.

Extension logs: Lambda sends the log output of extensions to CloudWatch Logs. Lambda also generates an additional log event for each extension during `Init`. The log event records the name and registration preference (event, config) on success, or the failure reason on failure.

Troubleshooting extensions

- If a `Register` request fails, make sure that the `Lambda-Extension-Name` header in the `Register` API call contains the full file name of the extension.
- If the `Register` request fails for an internal extension, make sure that the request does not register for the `Shutdown` event.

Extensions API reference

The OpenAPI specification for the extensions API version **2020-01-01** is available here: extensions-api.zip

You can retrieve the value of the API endpoint from the `AWS_LAMBDA_RUNTIME_API` environment variable. To send a `Register` request, use the prefix `2020-01-01/` before each API path. For example:

```
http://${AWS_LAMBDA_RUNTIME_API}/2020-01-01/extension/register
```

API methods

- Register
- Next
- Init error
- Exit error

Register

During `Extension init`, all extensions need to register with Lambda to receive events. Lambda uses the full file name of the extension to validate that the extension has completed the bootstrap sequence. Therefore, each `Register` API call must include the `Lambda-Extension-Name` header with the full file name of the extension.

Internal extensions are started and stopped by the runtime process, so they are not permitted to register for the `Shutdown` event.

Path – `/extension/register`

Method – **POST**

Request headers

- `Lambda-Extension-Name` – The full file name of the extension. Required: yes. Type: string.
- `Lambda-Extension-Accept-Feature` – Use this to specify optional Extensions features during registration. Required: no. Type: comma separated string. Features available to specify using this setting:
  - `accountId` – If specified, the Extension registration response will contain the account ID associated with the Lambda function that you're registering the Extension for.

Request body parameters

- `events` – Array of the events to register for. Required: no. Type: array of strings. Valid strings: `INVOKE`, `SHUTDOWN`.

> **Note**
>
> **Lambda Managed Instances:** Extensions for Lambda Managed Instances functions can only register for the `SHUTDOWN` event. Attempting to register for the `INVOKE` event will result in an error. This is because Lambda Managed Instances supports concurrent invocations within a single execution environment.

Response headers

- `Lambda-Extension-Identifier` – Generated unique agent identifier (UUID string) that is required for all subsequent requests.

Response codes

- 200 – Response body contains the function name, function version, and handler name.
- 400 – Bad Request
- 403 – Forbidden
- 500 – Container error. Non-recoverable state. Extension should exit promptly.

Example request body

```
{
    'events': [ 'INVOKE', 'SHUTDOWN']
}
```

Example response body

```
{
    "functionName": "helloWorld",
    "functionVersion": "$LATEST",
    "handler": "lambda_function.lambda_handler"
}
```

Example response body with optional accountId feature

```
{
    "functionName": "helloWorld",
    "functionVersion": "$LATEST",
    "handler": "lambda_function.lambda_handler",
    "accountId": "123456789012"
}
```

Next

Extensions send a `Next` API request to receive the next event, which can be an `Invoke` event or a `Shutdown` event. The response body contains the payload, which is a JSON document that contains event data.

The extension sends a `Next` API request to signal that it is ready to receive new events. This is a blocking call.

Do not set a timeout on the GET call, as the extension can be suspended for a period of time until there is an event to return.

Path – `/extension/event/next`

Method – **GET**

Request headers

- `Lambda-Extension-Identifier` – Unique identifier for extension (UUID string). Required: yes. Type: UUID string.

Response headers

- `Lambda-Extension-Event-Identifier` – Unique identifier for the event (UUID string).

Response codes

- 200 – Response contains information about the next event (`EventInvoke` or `EventShutdown`).
- 403 – Forbidden
- 500 – Container error. Non-recoverable state. Extension should exit promptly.

Init error

The extension uses this method to report an initialization error to Lambda. Call it when the extension fails to initialize after it has registered. After Lambda receives the error, no further API calls succeed. The extension should exit after it receives the response from Lambda.

Path – `/extension/init/error`

Method – **POST**

Request headers

- `Lambda-Extension-Identifier` – Unique identifier for extension. Required: yes. Type: UUID string.
- `Lambda-Extension-Function-Error-Type` – Error type that the extension encountered. Required: yes. This header consists of a string value. Lambda accepts any string, but we recommend a format of <category.reason>. For example:
  - Extension.NoSuchHandler
  - Extension.APIKeyNotFound
  - Extension.ConfigInvalid
  - Extension.UnknownReason

Request body parameters

- `ErrorRequest` – Information about the error. Required: no.

This field is a JSON object with the following structure:

```
{
      errorMessage: string (text description of the error),
      errorType: string,
      stackTrace: array of strings
}
```

Note that Lambda accepts any value for `errorType`.

The following example shows a Lambda function error message in which the function could not parse the event data provided in the invocation.

Example Function error

```
{
      "errorMessage" : "Error parsing event data.",
      "errorType" : "InvalidEventDataException",
      "stackTrace": [ ]
}
```

Response codes

- 202 – Accepted
- 400 – Bad Request
- 403 – Forbidden
- 500 – Container error. Non-recoverable state. Extension should exit promptly.

Exit error

The extension uses this method to report an error to Lambda before exiting. Call it when you encounter an unexpected failure. After Lambda receives the error, no further API calls succeed. The extension should exit after it receives the response from Lambda.

Path – `/extension/exit/error`

Method – POST

Request headers

- `Lambda-Extension-Identifier` – Unique identifier for extension. Required: yes. Type: UUID string.
- `Lambda-Extension-Function-Error-Type` – Error type that the extension encountered. Required: yes. This header consists of a string value. Lambda accepts any string, but we recommend a format of <category.reason>. For example:
  - Extension.NoSuchHandler
  - Extension.APIKeyNotFound
  - Extension.ConfigInvalid
  - Extension.UnknownReason

Request body parameters

- `ErrorRequest` – Information about the error. Required: no.

This field is a JSON object with the following structure:

```
{
      errorMessage: string (text description of the error),
      errorType: string,
      stackTrace: array of strings
}
```

Note that Lambda accepts any value for `errorType`.

The following example shows a Lambda function error message in which the function could not parse the event data provided in the invocation.

Example Function error

```
{
      "errorMessage" : "Error parsing event data.",
      "errorType" : "InvalidEventDataException",
      "stackTrace": [ ]
}
```

Response codes

- 202 – Accepted
- 400 – Bad Request
- 403 – Forbidden
- 500 – Container error. Non-recoverable state. Extension should exit promptly.